THE AGENT BLUEPRINT · PDF GUIDE

Stop your AI agent from overbuilding

The 5-step blueprint

AI coding agents tend to overcomplicate what they build. Five simple steps to keep it clean, with prompts and templates you can copy.

01 **Plan before it builds**

02 **Decide who does what**

03 **Fewer, clearer tools**

04 **Write the rules down**

05 **Ask it to clean up**

Dhruv Patel
For anyone building with AI coding agents

solwise.ai

BEFORE YOU START

Why agents overbuild

AI coding agents are very good at following instructions. Ask for a feature and you'll get one, usually fast.

The catch is that agents tend to solve problems by adding something: a new file, a new function, a new helper. They rarely stop on their own to ask whether the structure still makes sense, or whether something could be removed instead.

One addition is fine. Weeks of them, and the project gets harder to read and harder to change, for you and for the agent.

WARNING SIGNS

! Too many tools

Several tools do almost the same thing, and the agent keeps picking the wrong one.

! The same logic in several places

You fix a bug in one spot and it's still there in another.

! Files nobody asked for

New helpers, wrappers and config files show up that the task didn't need.

! Every fix adds more code

Problems get solved by stacking code on top, never by removing or rethinking.

The agent is doing what it was asked. So the fix is on our side: ask for different things, at the right moments. That's what the 5 steps are for.

THE 5 STEPS

01 **Plan before it builds**

02 **Decide who does what**

03 **Fewer, clearer tools**

04 **Write the rules down**

05 **Ask it to clean up**

STEP 1 OF 5

01 Plan before it builds

Before the agent writes anything, ask it what it plans to do. Read the plan, push back on anything bigger than the task, then approve it.

Reading a plan takes a few minutes. Untangling code you never wanted takes a lot longer.

When you read the plan, look out for new files, new dependencies (outside libraries or packages the code will rely on) and anything that sounds like "in case we need it later".

CLAUDE CODE

Claude Code has a **plan mode**. In plan mode, Claude looks through your code and proposes a plan without editing any files, and waits for you to approve it. In any other agent, use the prompt below.

PLAN-FIRST PROMPT — Copy, paste, fill in the task

```
Before you write any code, I want a plan.

The task: [describe what you want built or changed]

Reply with:
1. Your understanding of the task, in 2 or 3 sentences.
2. The files you plan to create or change, and what each one is for.
3. Any new tools, functions or dependencies you want to add, and why each one is needed.
4. Anything you are unsure about. Ask me instead of guessing.
5. The simplest version that would work. If you think something more complex is needed,
explain why.

Do not write or change any code yet. Wait until I reply with "approved" or with changes to
the plan.
```

STEP 2 OF 5

02 Decide who does what

When you build an AI agent, some jobs belong to the agent and some belong to the system around it: your database, your files and your regular code.

A simple rule: **the system keeps track of data and state, and the agent asks for what it needs.** State means where things currently stand, like which step an order is on or which tasks are done.

When an agent has to remember things itself, it starts building its own copies: extra variables, notes, little tracking files. Those copies drift out of date, and then you have two versions of the truth.

Some jobs belong to a person. Anything with real consequences, like money, deleting data or messages to customers, should get a human sign-off until you trust the setup.

RESPONSIBILITY MAP · EXAMPLE: AN AGENT THAT ANSWERS QUESTIONS ABOUT ORDERS

RESPONSIBILITY	OWNED BY (system, agent, or person)	WHY
Storing the status of each order	System	The database is the single source of truth. The agent looks the status up and never guesses or keeps its own copy.
Writing the reply to the customer	Agent	Turning order details into a clear, friendly answer is language work.
Approving refunds above a set amount	Person	Money decisions need a human sign-off.

A blank version is in the starter kit: responsibility-map.md

STEP 3 OF 5

03 Fewer, clearer tools

A tool is a function an AI agent can call to do something: look up a customer, send an email, create a ticket. Each time the agent acts, it picks one of its tools.

More tools means more choices, and more chances to pick the wrong one. Coding agents also tend to add a new tool for every small need, so the list grows quietly.

FOUR RULES

1 One job per tool

If you need "and" to describe it, it's probably two tools.

2 No two tools doing the same job

Overlap makes the agent guess which one to use.

3 Clear names

`get_customer` tells you what it does. `fetch_data_v2` doesn't.

4 Say when to use it

The description is what the agent reads when it decides which tool to pick.

BEFORE · 3 TOOLS

```
get_customer_email(customer_id)
get_customer_phone(customer_id)
get_customer_address(customer_id)
```

→

AFTER · 1 TOOL

`get_customer(customer_id)`

Look up one customer by their ID. Use this whenever you need any customer details. Returns name, email, phone and address.

The agent has fewer choices, and there's one place to fix if customer data changes.

To audit an existing project, use the tool review prompt in the starter kit: prompts.md

STEP 4 OF 5

04 Write the rules down

By default, an agent doesn't carry your earlier conversations into a new session. Whatever you explained yesterday, you'd have to explain again.

A project rules file fixes this. It's a plain text file in your project that the agent reads at the start of each session. Write your rules once and they apply every time.

MD AGENTS.md

A common convention that several coding agents read.

MD CLAUDE.md

The file Claude Code reads. The content can be the same. Only the name changes.

Both use Markdown, a simple text format where `#` makes a heading and `-` makes a bullet point. Keep the file short and specific. Long, vague rules are easy to miss.

A SHORT EXAMPLE

AGENTS.md

```
# AGENTS.md

## About this project
- An app that tracks customer orders. Python with FastAPI.
- Run the tests with: pytest

## How to work
- Plan first. Wait for my approval before writing code.
- Ask when unsure instead of guessing.
- Pick the simplest solution that works.

## Code rules
- Reuse existing functions before writing new ones.
- Don't add new dependencies without asking.
- When you replace code, delete the old version.
```

Full templates for both files are in the starter kit.

STEP 5 OF 5

05 Ask it to clean up

Agents are good at adding things and rarely suggest removing them. In practice, an agent won't go back and simplify code that already works unless you ask.

So ask. After each feature, run a simplify pass: the agent reviews what was built and suggests what to merge, remove or cut back. You approve the changes you want. Doing it after each feature keeps each pass small.

SIMPLIFY-PASS PROMPT — Copy and paste after each feature

```
We just finished a feature. Before we move on, do a simplify pass on the code we changed and
the code around it. Do not change anything yet.

Look for:
1. Tools or functions that do the same or overlapping jobs and could be merged.
2. Logic that is repeated in more than one place.
3. Code, files or dependencies that nothing uses anymore.
4. Over-engineering: extra layers, settings or abstractions that the current feature does
not need.

For each thing you find, tell me:
- Where it is (file and function).
- What you suggest.
- What could break, and how we would check.

Put the safest and most useful changes first. Then stop and wait. Only make the changes I
approve, and run the tests after each one.
```

WHAT TO CHECK IN THE ANSWER

- [ ] Tools that overlap
- [ ] Logic repeated in more than one place
- [ ] Code, files or dependencies nothing uses
- [ ] Layers or settings the feature doesn't need
- [ ] Tests still pass after each change

CLAUDE CODE The starter kit includes a **simplify-pass skill**. A skill is a folder with a SKILL.md file of instructions that Claude can follow. Add it to your project, then ask Claude to "do a simplify pass".

PRINT THIS PAGE

Checklist

Run through it for every feature you build with an agent.

01 Plan before it builds

- [ ] I asked for a plan before any code was written.
- [ ] I cut anything bigger than the task, then approved the plan.

02 Decide who does what

- [ ] The system keeps track of data and state. The agent asks for it.
- [ ] Every job in the responsibility map has an owner.

03 Fewer, clearer tools

- [ ] Each tool does one job, and no two tools overlap.
- [ ] Every tool has a clear name and says when to use it.

04 Write the rules down

- [ ] The project has an AGENTS.md or CLAUDE.md.
- [ ] It says how to run the tests and includes the plan-first rule.

05 Ask it to clean up

- [ ] I ran a simplify pass after this feature.
- [ ] I approved the changes, and the tests still pass.

Questions or feedback: `solwise.ai`